7CS034 Lab 3 Tidying & Importing Data

1 Objectives

By the end of this session we will be able to

- Import, transform and wrangle data files in `Rstudio` until they are in a tidy format.
- Understand the other data tools from `readr`.

2 Let's get started

Let's start by working through the code we used during the lecture this week. Copy and paste each piece of code and ensure that you understand what we covered. The lecture focused on some options for tidying data and it was quite technical in places. In this lab session we are going to learn about importing data. As usual, we are following R for Data Science and you are welcome to go there and work along with the relevant chapter.

3 Importing Data

Specifically, this section will focus on reading plain-text rectangular files. We'll start with practical advice for handling features like column names, types, and missing data. You will then learn about reading data from multiple files at once and writing data from R to a file. Finally, you'll learn how to handcraft data frames in R. We'll be using the `readr` command which is part of the `tidyverse`.

```
library(tidyverse)
```

4 Reading data from a file

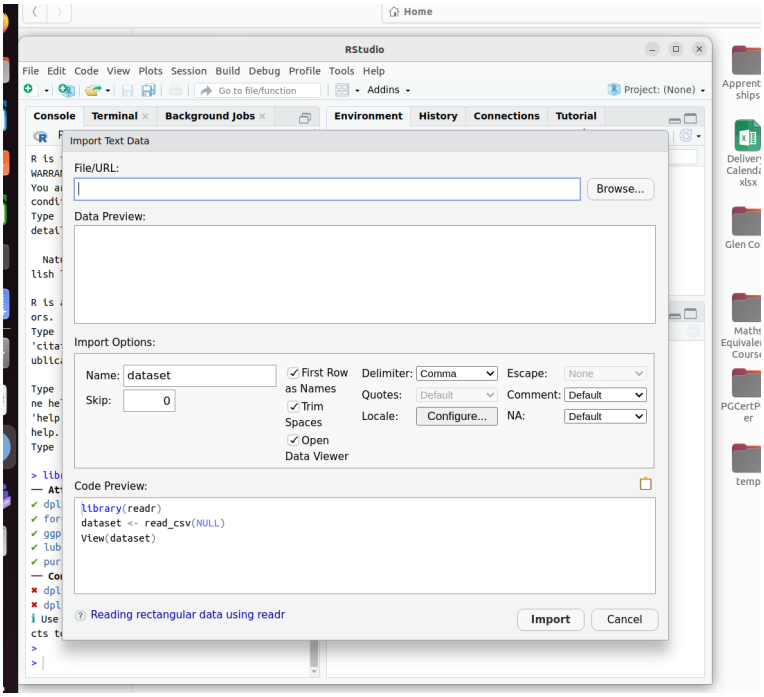
To begin, we'll focus on the most common rectangular data file type: CSV, which is short for comma-separated values. Here is what a simple CSV file looks like. The first row, commonly called the header row, gives the column names, and the following six rows provide the data. The columns are separated, aka delimited, by commas.

```
Student ID,Full Name,favourite.food,mealPlan,AGE
1,Sunil Huffmann,Strawberry yoghurt,Lunch only,4
2,Barclay Lynn,French fries,Lunch only,5
3,Jayendra Lyne,N/A,Breakfast and lunch,7
4,Leon Rossini,Anchovies,Lunch only,
5,Chidiegwu Dunkel,Pizza,Breakfast and lunch,five
6,Güvenç Attila,Ice cream,Lunch only,6
```

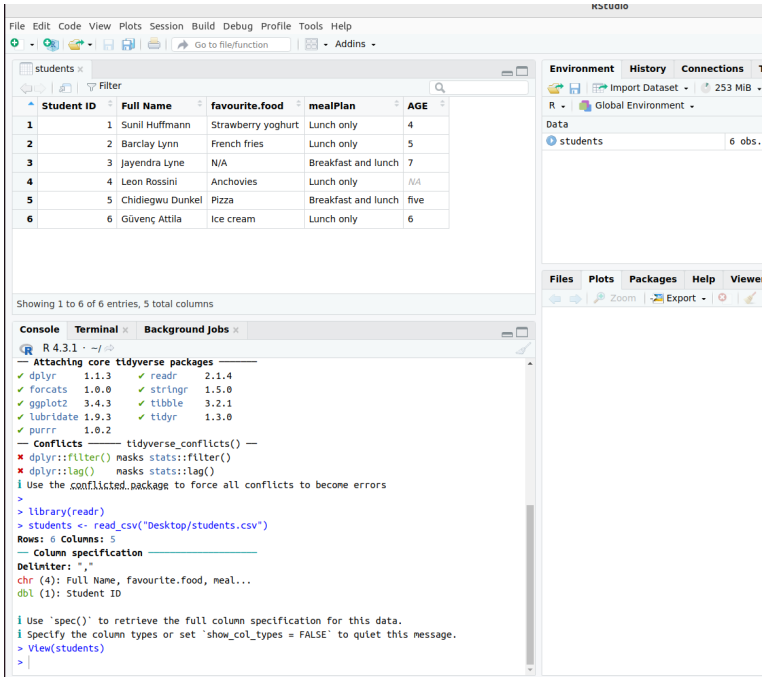
We can read this file into `R` using `read_csv()`. The first argument is the most important: **the path to the file**. You can think about the path as the address of the file: the file is called students.csv and that it lives on my desktop. To do so we can use the menu in `Rstudio`.

- Click **File**.
- Click **Import Dataset**.
- Click **From Text (readr)**.

For me this brings up the window below where I use **Browse** to find the file.



If everythig goes to plan you'll see something like this



This also shows you the correct `readr` statement needed to read the file from whatever directory you saved it in. For me this is

```
students <- read_csv("Desktop/students.csv")
```

You can also download the `students.csv` file from https://pos.it/r4ds-students-csv or you can read it directly from that URL with:

```
students <- read_csv("https://pos.it/r4ds-students-csv")
```

This is very useful since it means that we can read data from online repositories (which is often where real world data is naturally stored). When you run `read_csv()`, it prints out a message telling you the number of rows and columns of data, the delimiter that was used, and the column specifications (names of columns organized by the type of data the column contains). It also prints out some information about retrieving the full column specification and how to quiet this message. This message is an integral part of `readr`.

4.1 Practical Advice

Once you read data in, the first step usually involves transforming it in some way to make it easier to work with in the rest of your analysis. Let's take another look at the `students` data with that in mind.

```
> students
# A tibble: 6 × 5
  `Student ID` `Full Name`      favourite.food     mealPlan            AGE
         <dbl> <chr>            <chr>              <chr>               <chr>
1            1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
2            2 Barclay Lynn     French fries       Lunch only          5
3            3 Jayendra Lyne    N/A                Breakfast and lunch 7
4            4 Leon Rossini     Anchovies          Lunch only          NA
5            5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
6            6 Güvenç Attila    Ice cream          Lunch only          6
>
```

In the `favourite.food` column, there are a bunch of food items, and then the character string `N/A`, which should have been a real `NA` that `R` will recognize as "not available". This is something we can address using the `na` argument. By default, `read_csv()` only recognizes empty strings (`""`) in this dataset as `NA`s, we want it to also recognize the character string "`N/A`".

```
students <- read_csv("Desktop/students.csv", na = c("N/A", ""))
```

```
> students <- read_csv("Desktop/students.csv", na = c("N/A", ""))
Rows: 6 Columns: 5
── Column specification ────────────────────────
Delimiter: ","
chr (4): Full Name, favourite.food, mealP...
dbl (1): Student ID

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
>
```

You might also notice that the `Student ID` and `Full Name` columns are surrounded by backticks. That's because they contain spaces, breaking `R`'s usual rules for variable names; they're non-syntactic names. To refer to these variables, you need to surround them with backticks, ‘:

```
students |>
  rename(
    student_id = ‘Student ID‘,
    full_name = ‘Full Name‘
  )
```

```
> students |>
+     rename(
+         student_id = `Student ID`,
+         full_name = `Full Name`
+     )
# A tibble: 6 × 5
  student_id full_name        favourite.food
       <dbl> <chr>            <chr>
1          1 Sunil Huffmann   Strawberry yog…
2          2 Barclay Lynn     French fries
3          3 Jayendra Lyne    NA
4          4 Leon Rossini     Anchovies
5          5 Chidiegwu Dunkel Pizza
6          6 Güvenç Attila    Ice cream
# ℹ 2 more variables: mealPlan <chr>,
#   AGE <chr>
>
```

An alternative approach is to use `janitor::clean_names()` from the `janitor` package to use some heuristics to turn them all into so called snake case at once.

```
students |> janitor::clean_names()
```

```
> students |> janitor::clean_names()
# A tibble: 6 × 5
  student_id full_name        favourite_food     meal_plan           age
       <dbl> <chr>            <chr>              <chr>               <chr>
1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
2          2 Barclay Lynn     French fries       Lunch only          5
3          3 Jayendra Lyne    NA                 Breakfast and lunch 7
4          4 Leon Rossini     Anchovies          Lunch only          NA
5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
6          6 Güvenç Attila    Ice cream          Lunch only          6
> |
```

Another common task after reading in data is to consider variable types. For example, `meal_plan` is a categorical variable with a known set of possible values, which in `R` should be represented as a factor:

```
students |>
  janitor::clean_names() |>
  mutate(meal_plan = factor(meal_plan))
```

```
> students |>
+     janitor::clean_names() |>
+     mutate(meal_plan = factor(meal_plan))
# A tibble: 6 × 5
  student_id full_name        favourite_food     meal_plan           age
       <dbl> <chr>            <chr>              <fct>               <chr>
1          1 Sunil Huffmann   Strawberry yoghurt Lunch only          4
2          2 Barclay Lynn     French fries       Lunch only          5
3          3 Jayendra Lyne    NA                 Breakfast and lunch 7
4          4 Leon Rossini     Anchovies          Lunch only          NA
5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch five
6          6 Güvenç Attila    Ice cream          Lunch only          6
> |
```

Note that the values in the `meal_plan` variable have stayed the same, but the type of variable denoted underneath the variable name has changed from character (`<chr>`) to factor (`<fct>`). You'll learn more about factors later.

Before you analyze this data, you'll probably want to fix the `age` and `id` columns. Currently, `age` is a character variable because one of the observations is typed out as five instead of a numeric 5.

```
students <- students |>
  janitor::clean_names() |>
  mutate(
    meal_plan = factor(meal_plan),
    age = parse_number(if_else(age == "five", "5", age))
  )
```

```
> students <- students |>
+     janitor::clean_names() |>
+     mutate(
+         meal_plan = factor(meal_plan),
+         age = parse_number(if_else(age == "five", "5", age))
+     )
> students
# A tibble: 6 × 5
  student_id full_name        favourite_food     meal_plan             age
       <dbl> <chr>            <chr>              <fct>               <dbl>
1          1 Sunil Huffmann   Strawberry yoghurt Lunch only              4
2          2 Barclay Lynn     French fries       Lunch only              5
3          3 Jayendra Lyne    NA                 Breakfast and lunch     7
4          4 Leon Rossini     Anchovies          Lunch only             NA
5          5 Chidiegwu Dunkel Pizza              Breakfast and lunch     5
6          6 Güvenç Attila    Ice cream          Lunch only              6
> |
```

A new function here is `if_else()`, which has three arguments. The first argument `test` should be a logical vector. The result will contain the value of the second argument, `yes`, when `test` is `TRUE`, and the value of the third argument, `no`, when it is `FALSE`. Here we're saying if `age` is the character string "`five`", make it "`5`", and if not leave it as `age`.

4.2 Other arguments

There are a couple of other important arguments that we need to mention, and they'll be easier to demonstrate if we first show you a handy trick: `read_csv()` can read text strings that you've created and formatted like a CSV file:

```
read_csv(
  "a,b,c
  1,2,3
  4,5,6"
)
```

```
> read_csv(
+     "a,b,c
+   1,2,3
+   4,5,6"
+ )
Rows: 2 Columns: 3
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
dbl (3): a, b, c

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
# A tibble: 2 × 3
      a     b     c
  <dbl> <dbl> <dbl>
1     1     2     3
2     4     5     6
> |
```

Usually, `read_csv()` uses the first line of the data for the column names, which is a very common convention. But it's not uncommon for a few lines of metadata to be included at the top of the file. You can use `skip = n` to skip the first `n` lines or use `comment = "#"` to drop all lines that start with (e.g.) `#`:

```
read_csv(
```

```
  "The first line of metadata
  The second line of metadata
  x,y,z
  1,2,3",
  skip = 2
)
```

```
> read_csv(
+     "The first line of metadata
+   The second line of metadata
+   x,y,z
+   1,2,3",
+     skip = 2
+ )
Rows: 1 Columns: 3
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
dbl (3): x, y, z

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
# A tibble: 1 × 3
      x     y     z
  <dbl> <dbl> <dbl>
1     1     2     3
>
```

```
read_csv(
  "# A comment I want to skip
  x,y,z
  1,2,3",
  comment = "#"
)
```

```
> read_csv(
+     "# A comment I want to skip
+   x,y,z
+   1,2,3",
+     comment = "#"
+ )
Rows: 1 Columns: 3
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
dbl (3): x, y, z

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
# A tibble: 1 × 3
      x     y     z
  <dbl> <dbl> <dbl>
1     1     2     3
>
```

In other cases, the data might not have column names. You can use `col_names = FALSE` to tell `read_csv()` not to treat the first row as headings and instead label them sequentially from `X1` to `Xn`:

```
read_csv(
  "1,2,3
  4,5,6",
  col_names = FALSE
)
```

```
> read_csv(
+     "# A comment I want to skip
+   x,y,z
+   1,2,3",
+     comment = "#"
+ )
Rows: 1 Columns: 3
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
dbl (3): x, y, z

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
# A tibble: 1 × 3
      x     y     z
  <dbl> <dbl> <dbl>
1     1     2     3
>
```

Alternatively, you can pass `col_names` a character vector which will be used as the column names:

```
read_csv(
  "1,2,3
  4,5,6",
  col_names = c("x", "y", "z")
)
```

```
> read_csv(
+     "1,2,3
+   4,5,6",
+     col_names = c("x", "y", "z")
+ )
Rows: 2 Columns: 3
── Column specification ────────────────────────────────────────────────────────
Delimiter: ","
dbl (3): x, y, z

ℹ Use `spec()` to retrieve the full column specification for this data.
ℹ Specify the column types or set `show_col_types = FALSE` to quiet this message.
# A tibble: 2 × 3
      x     y     z
  <dbl> <dbl> <dbl>
1     1     2     3
2     4     5     6
>
```

These arguments are all you need to know to read the majority of CSV files that you'll encounter in practice. (For the rest, you'll need to carefully inspect your .csv file and read the documentation for `read_csv()`'s many other arguments.)

Once you've mastered `read_csv()`, using `readr`'s other functions is straightforward; it's just a matter of knowing which function to reach for. Check out the cheat sheet at https://readr.tidyverse.org.

4.3 Exercises

Q1. What function would you use to read a file where fields were separated with "|"?

Q2. Apart from `file`, `skip`, and `comment`, what other arguments do `read_csv()` and `read_tsv()` have in common?

Q3. What are the most important arguments to `read_fwf()`?

Q4. Sometimes strings in a CSV file contain commas. To prevent them from causing problems, they need to be surrounded by a quoting character, like `"` or `'`. By default, `read_csv()` assumes that the quoting character will be `"`. To read the following text into a data frame, what argument to `read_csv()` do you need to specify?

```
"x,y\n1,'a,b'"
```

Q5. Identify what is wrong with each of the following inline CSV files. What happens when you run the code?

```
read_csv("a,b\n1,2,3\n4,5,6")
read_csv("a,b,c\n1,2\n1,2,3,4")
read_csv("a,b\n\"1")
read_csv("a,b\n1,2\na,b")
read_csv("a;b\n1;3")
```

That's enough for today but you are directed to read and work through the rest of the material here before next week.

https://r4ds.hadley.nz/data-import